

Overview of Statistical Learning and Modeling – 36-600

Lab 9R: Random Forests and Boosting

Due Friday, October 31, 11:59pm

Trishla Nair

Regression

First, let's import the heart-disease dataset and log-transform the response variable, **Cost**:

```
suppressMessages(library(tidyverse))
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
df      <- read.csv("https://raw.githubusercontent.com/FSKoerner/F25-36600-data/refs/heads/main/heart_d_1.csv")
df      <- df[, -10]
w       <- which(df$Cost > 0)
df      <- df[w, ]
df$Cost <- log(df$Cost)
```

Question 1

Split these data into training and test sets, reusing the random-number-generator seed you used in previous labs when analyzing these data.

```
set.seed(42)
s       <- sample(nrow(df), round(0.7 * nrow(df)))
df.train <- df[s, ]
df.test  <- df[-s, ]
```

Question 2

Learn a random forest model on the training data, and compute the MSE. Remember to set **importance = TRUE**. *Note:* for reproducible results, set the seed again before fitting the random forest! Assuming you split the data in the same manner as you did before, feel free to look back at your other labs and see if the MSE is smaller here. (For my split? It's about 10% smaller here than it was for a regression tree.)

```
suppressMessages(library(randomForest))
```

```
## Warning: package 'randomForest' was built under R version 4.4.3
```

```

set.seed(42)
rf.out <- randomForest(Cost ~ ., data = df.train, importance = TRUE)
resp.pred <- predict(rf.out, newdata = df.test)

actual_values <- df.test$Cost
mse <- mean((actual_values - resp.pred)^2)
print(mse)

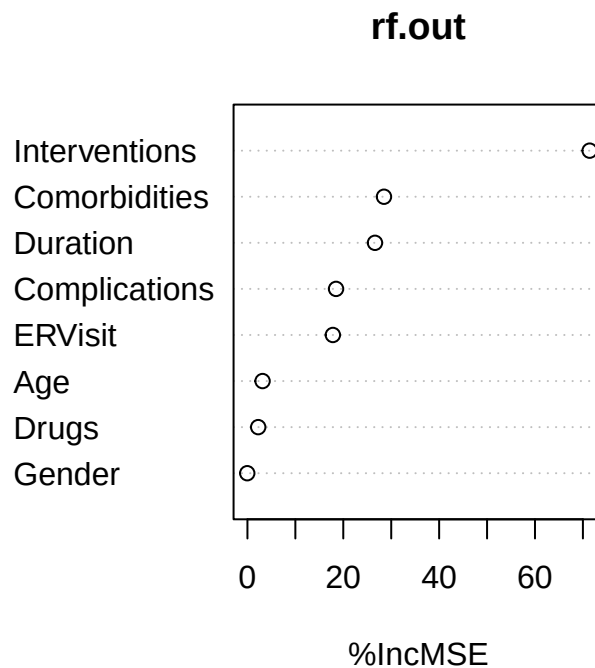
```

```
## [1] 1.305654
```

Question 3

Create a variable importance plot. Remember to pass `type = 1` as an argument to this plotting function. Mentally note the important variables. These should be consistent with those variables that appeared in your regression tree in the tree lab.

```
varImpPlot(rf.out, type = 1)
```



Question 4

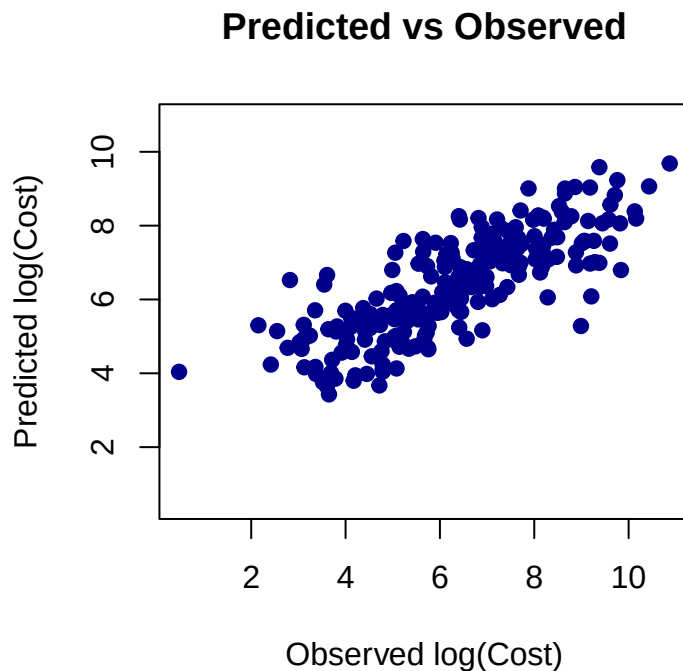
Show the diagnostic plot of predicted test-set response values vs. observed test-set response values. As usual, make sure the limits are the same along both axes and plot the diagonal $y = x$ line.

```

plot(actual_values, resp.pred,
      xlab = "Observed log(Cost)",

```

```
ylab = "Predicted log(Cost)",
main = "Predicted vs Observed",
xlim = range(c(actual_values, resp.pred)),
ylim = range(c(actual_values, resp.pred)),
pch = 19, col = "darkblue")
```



Question 5

Now learn an extreme gradient boosting model, and show the test-set MSE. Note that in order to do this, we have to remove the variables `Gender`, `Drugs`, and `Complications`, which are factor or factor-like variables, and for ease of implementation, we'll first break up `df.train` and `df.test` into predictor and response variables (i.e., once you have the objects `df.train` and `df.test` defined, you should remove the `eval=FALSE` cell option below):

```
df.train %>% dplyr::select(., -Gender, -Drugs, -Complications) -> df.train
df.test %>% dplyr::select(., -Gender, -Drugs, -Complications) -> df.test
resp.train <- df.train[, 1]
resp.test <- df.test[, 1]
pred.train <- df.train[, -1]
pred.test <- df.test[, -1]
```

Note that as a result of discarding some predictors, the MSE that we get might not be as good as it was for the random forest. But we'll see!

```
suppressMessages(library(xgboost))
```

```
## Warning: package 'xgboost' was built under R version 4.4.3
```

```

train <- xgb.DMatrix(data = as.matrix(pred.train), label = resp.train)
test  <- xgb.DMatrix(data = as.matrix(pred.test),  label = resp.test)

xgb.cv.out <- xgb.cv(
  params = list(objective = "reg:squarederror"),
  data = train,
  nrounds = 30,
  nfold = 5,
  verbose = 0
)

rmse.vec <- xgb.cv.out$evaluation_log$test_rmse_mean

xgb.out <- xgboost(
  data = train,
  nrounds = which.min(rmse.vec),
  params = list(objective = "reg:squarederror"),
  verbose = 0
)

resp.pred <- predict(xgb.out, newdata = test)

mse <- mean((resp.pred - resp.test)^2)

print(mse)

```

```
## [1] 1.427078
```

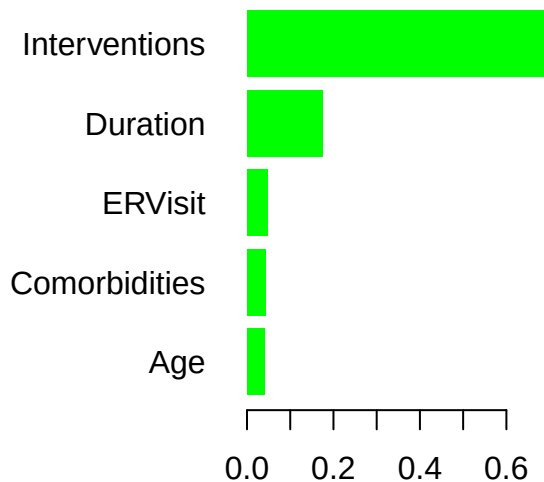
Question 6

Create a variable importance plot for the extreme gradient boosting model. Make a mental note about whether the variables identified here as important are the same or similar to those identified by the random forest.

```

imp.out <- xgb.importance(model = xgb.out)
xgb.plot.importance(importance_matrix = imp.out,
  col = "green")

```



Classification

Let's now load in the data on political movements, which you've looked at in previous labs:

```
load(url("https://github.com/FSKoerner/F25-36600-data/raw/refs/heads/main/movement.Rdata"))
f <- function(variable, level0 = "NO", level1 = "YES") {
  n <- length(variable)
  new.variable <- rep(level0, n)
  w <- which(variable == 1)
  new.variable[w] <- level1
  return(factor(new.variable))
}
predictors$nonviol <- f(predictors$nonviol)
predictors$sanctions <- f(predictors$sanctions)
predictors$aid <- f(predictors$aid)
predictors$support <- f(predictors$support)
predictors$viol.repress <- f(predictors$viol.repress)
predictors$defect <- f(predictors$defect)
levels(response) <- c("FAILURE", "SUCCESS")
df <- cbind(predictors, response)
names(df)[9] <- "label"
rm(id.half, id, predictors.half, predictors, response)
```

Note that given the number of factor variables in this dataset, we'll forego learning a boosting model below.

Question 7

Split the data! Recreate what you did for previous labs, including the random-number-generator seed.

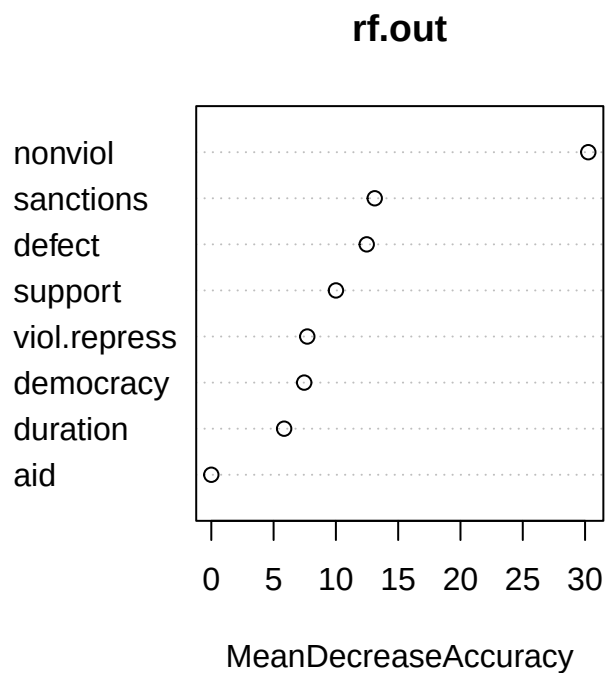
```
set.seed(42)
s      <- sample(nrow(df), round(0.7 * nrow(df)))
df.train <- df[s, ]
df.test  <- df[-s, ]
```

Question 8

Learn a random forest model. Output probabilities for Class 1 (see the class notes!), but do not output a confusion matrix or a misclassification rate. It will become clear why we're holding off on computing these quantities for now. However, do go ahead and create the variable importance plot here.

```
set.seed(42)
rf.out <- randomForest(label ~ ., data = df.train, importance = TRUE)
out.pred <- predict(rf.out, newdata = df.test, type = "prob")[, 2]

varImpPlot(rf.out, type = 1)
```



Question 9

Plot a ROC curve for the random forest, and output the AUC value.

```
library(pROC)
```

```
## Warning: package 'pROC' was built under R version 4.4.3
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

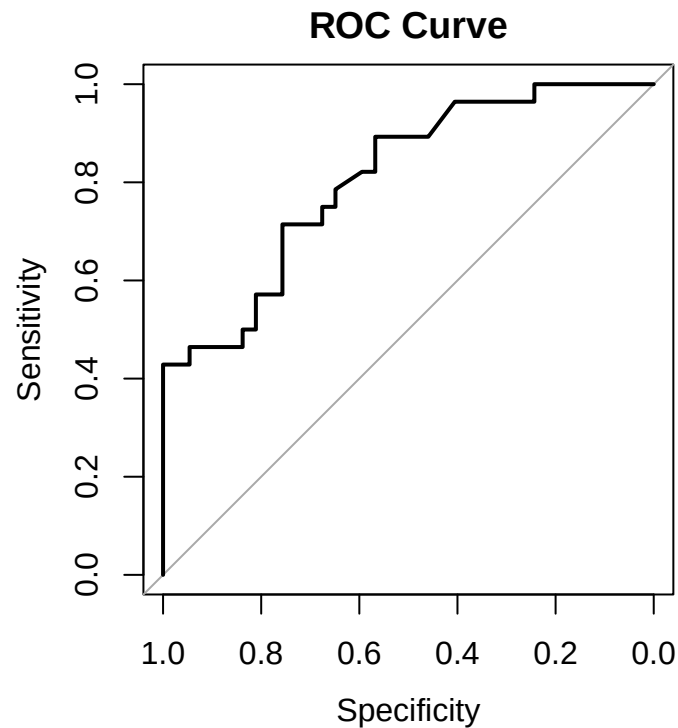
```
##      cov, smooth, var
```

```
actual_values <- df.test$label
```

```
roc.obj <- roc(actual_values, out.pred, levels = c("FAILURE", "SUCCESS"))
```

```
## Setting direction: controls < cases
```

```
plot(roc.obj, main = "ROC Curve")
```



```
auc.value <- auc(roc.obj)
```

```
cat("AUC:", round(auc.value, 3))
```

```
## AUC: 0.808
```

Question 10

Use Youden's J statistic to determine the optimal class-separation threshold. Output that number. Then, using that threshold, transform the test-set Class 1 probabilities to class predictions, and finally, now output the confusion matrix and calculate misclassification rate. (*Note*: feel free to reuse code from previous labs.)

```
roc.obj <- roc(actual_values, out.pred, levels = c("FAILURE", "SUCCESS"))
```

```
## Setting direction: controls < cases
```

```
opt.thresh <- coords(roc.obj, "best", ret = "threshold", best.method = "youden")
print(opt.thresh)
```

```
## threshold
## 1 0.365
```

```
class.pred <- ifelse(out.pred >= opt.thresh, "SUCCESS", "FAILURE")
```

```
conf.mat <- table(Predicted = out.pred, Actual = df.test$label)
```

```
mcr <- 1 - sum(diag(conf.mat)) / sum(conf.mat)
print(mcr)
```

```
## [1] 0.9846154
```